

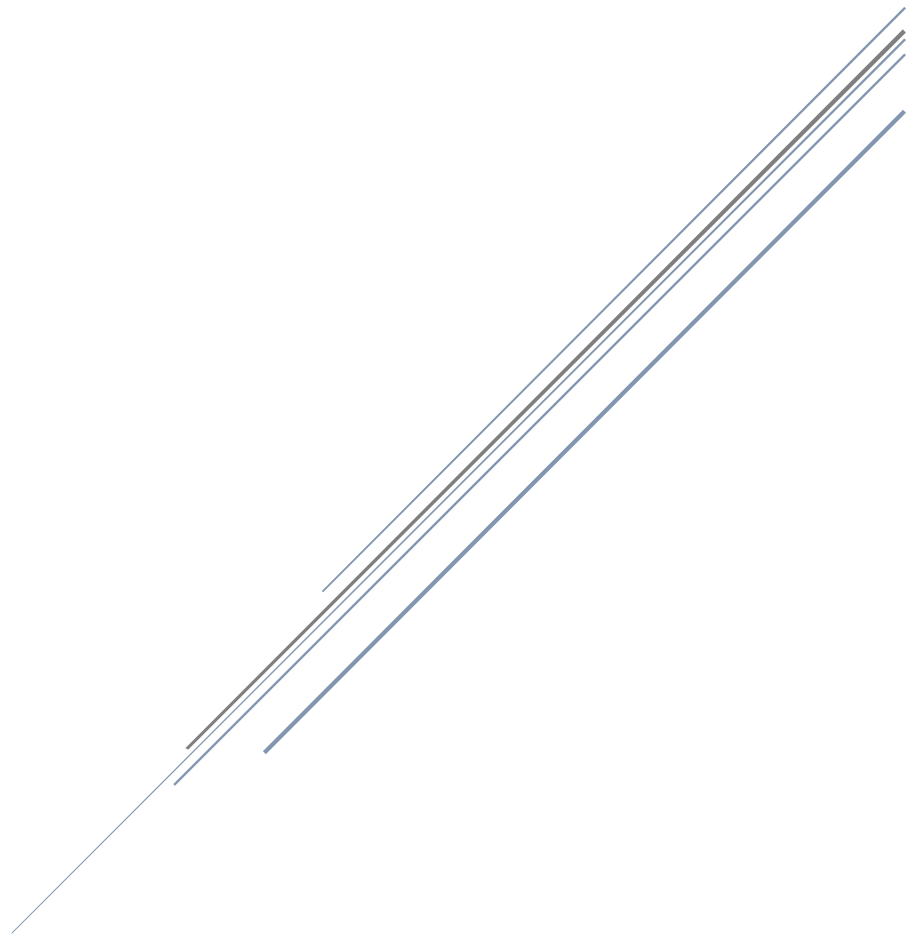


INDIVIDUAL PROJECT

INFORMATION SYSTEM APPLICATION

INSTRUCTOR: VO DINH HIEU

CLASS: CMU-IS 401 EIS



DA NANG ,15 November, 2025

FULL NAME	STUDENT ID
Hàng Gia Bảo	28211152592
Hoàng Đình Khánh	28219002146
Nguyễn Khắc Nguyên Khoa	28211149985
Huỳnh Trần Gia Huy	28211232063
Nguyễn Trung Kiên	28219005326
Nguyễn Ngọc Linh	28219050586

Document History		
Version	Date	Comments
V1.0	6/11	Create database, E-R Design
V2.0	8/11	Create DDL, DML
V3.0	10/11	Store Procedures
V4.0	11/11	Create Trigger
V5.0	12/11	Create Non-clustered Index
V6.0	15/11	Create Transaction and Set Isolation Level

TABLE OF CONTENT

1. Introduction	5
1.1. Purpose	5
1.2. Goal	5
1.3. Scope	5
1.4 Data storage platforms	5
1.5 Definition, Acronyms and Abbreviations	6
2. Database Design	6
2.1 Table Overview	6
2.2 Entity Relationship Diagram	7
2.3 Table Relationship Diagram	8
2.4 Detail	8
2.1.1 Account	8
2.1.2 StudentProfile	9
2.1.3 TeacherProfile	9
2.1.4 Groups	9
2.1.5 Student_Group	10
2.1.6 QuizzFolders	10
2.1.7 Quizzes	10
2.1.8 Quizz_Group	11
2.1.9 Questions	11
2.1.10 Options	11
2.1.11 OfflineResults	12
2.1.12 OfflineWrongAnswers	12
2.1.13 OfflineReports	12
3. Data Management and Implementation	13
3.1 Database design	13
3.1.1 Create Database and Tables	13
3.1.2 Insert Data	19
3.2 Procedures	24
3.2.1 Assign quiz to group with validation (Kien)	24
3.2.2 Submit Exam (Khoa)	26
3.2.3 Check-In Classroom Session (Linh)	26
3.2.4 Advanced Deep Clone Quiz With Questions And Options (Khanh)	28
3.2.5 Get Student Results By Quiz (Bao)	30
3.2.6 Add a new quiz(Huy)	30
3.3 Trigger	31
3.3.1 Update TotalParticipants when result is inserted (Kien)	31
3.3.2 Automatically Update Total Participants (Khoa)	32
3.3.3 Enforce Content Lock On Active Assignments (Khanh)	32
3.3.4 Validate Result Start And End Dates (Bao)	33
3.3.5 Auto Update Classroom Usage Statistics (Linh)	33
3.3.6 Increase total participants when an OfflineResult is created(Huy)	34
3.4 Index	35
3.4.1 Optimize student result lookups (Kien)	35
3.4.2 Composite Index for Exam History (Khoa)	35
3.4.3 Index For Group Names (Khanh)	35

3.4.4 Composite Index For Integrity Checks On Options (Bao)	35
3.4.5 Index OfflineResults(Huy)	35
3.4.6 Index for Appointment Filtering by Doctor & Date (Linh)	35
3.5 Isolated	36
3.5.1 Reading quiz data while submitting results(Kien)	36
3.5.2 Verifying student profile while account status is being updated (Khoa)	36
3.5.3 Serializable Transaction For Semester-End Archival (Khanh)	37
3.5.4 Read Uncommitted Quiz List (Bao)	38
3.5.5 Checking Product Stock While Processing Order(Linh)	38
3.5.6 Get Student Quiz Attempts (Huy)	39
3.6 Function	40
3.6.1 Calculate student's average score (Kien)	40
3.6.2 Check Student Remaining Attempts (Khoa)	41
3.6.3 Check If Quiz Is Private (Khanh)	41
3.6.4 Get Students Who Have Not Submitted (Bao)	42
3.6.5 Count total questions in a quiz (Huy)	42
3.6.6 Check If Student Has Overdue Quizzes (Linh)	43
4. Data Integrity and Security	44
4.1 Role:	44
4.2 Data control language (DCL)	46
5. Matrix Grade	48
6. References	48

1. Introduction

1.1. Purpose

This document describes the database design for the **VDH Online Quiz System**, detailing the logical and physical structure of its database. This Database Design document will introduce all entities and attributes of the System, which will help developers and testers implement and test based on this design.

1.2. Goal

To create accurate, efficient, and scalable database tables that ensure data integrity for the VDH system.

1.3. Scope

This Database Design Document provides the basics for the Database design. It describes both logical and physical definitions, non-functional issues, and the database interfaces. Storage aspects are defined in the physical database design sections . The following topics are covered in this document:

- Assumptions and decisions on database design .
- Entity-mapping.
- Table column definitions.
- Primary, unique, and foreign key definitions.
- Column and row-level validation rules (check constraints).
- Rules for populating specific columns (sequences, derivations, etc.).
- Interfaces and dependencies with other components .
- Data access description.

1.4 Data storage platforms

- Data from the application is stored in a **Microsoft SQL Server** database
- In SQL Server, the data structure is stored as tables. These database objects act as containers for the data, in which the data is logically organized in a rows-and-columns format

1.5 Definition, Acronyms and Abbreviations

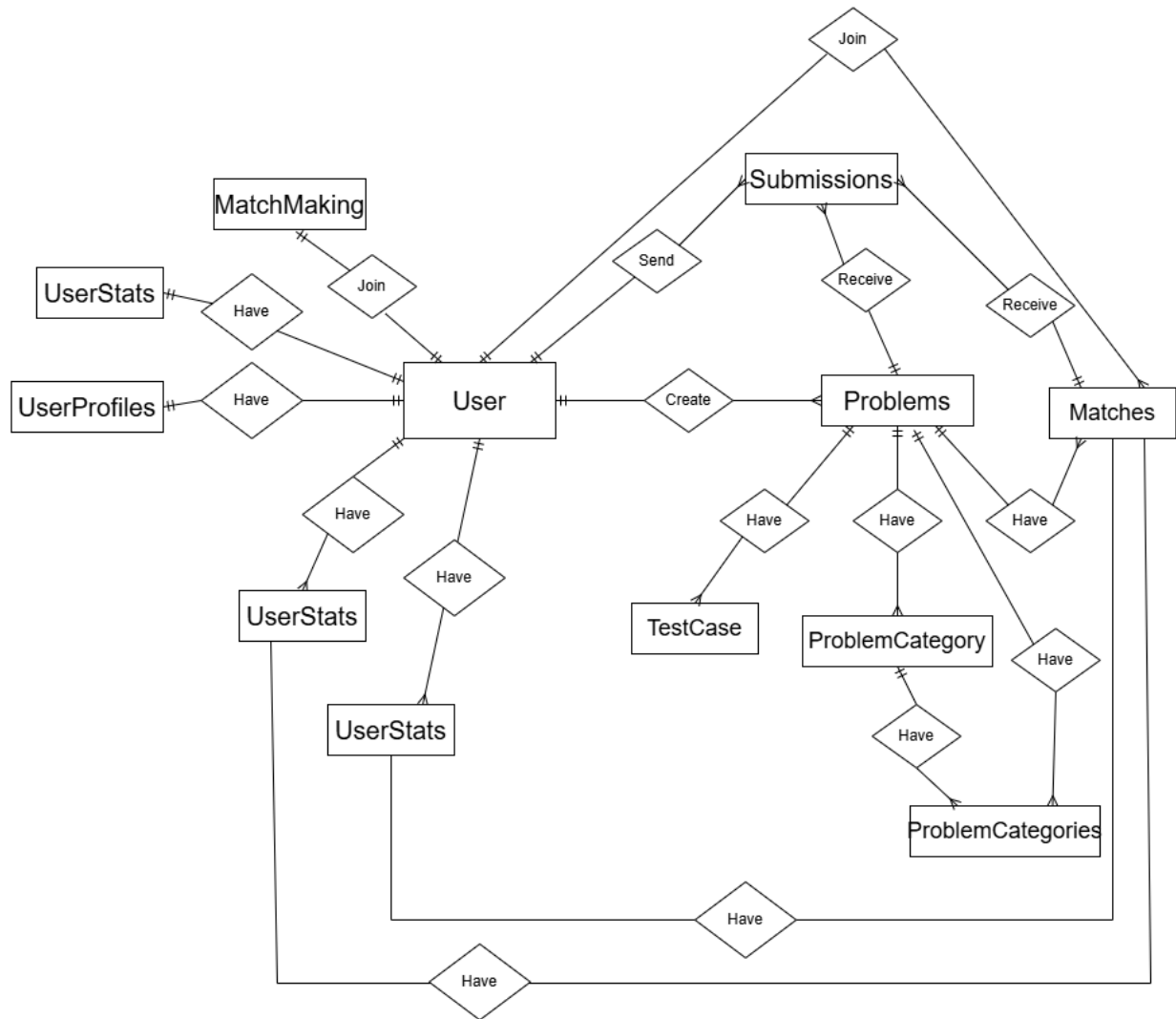
Abbreviations	Description	Comment
PK/FK	Primary/ Foreign Key	Use to indicate a file is a Primary or Foreign key in a table
ERD	Entity Relationship Diagram	Show the relationship between entities in the system

2. Database Design

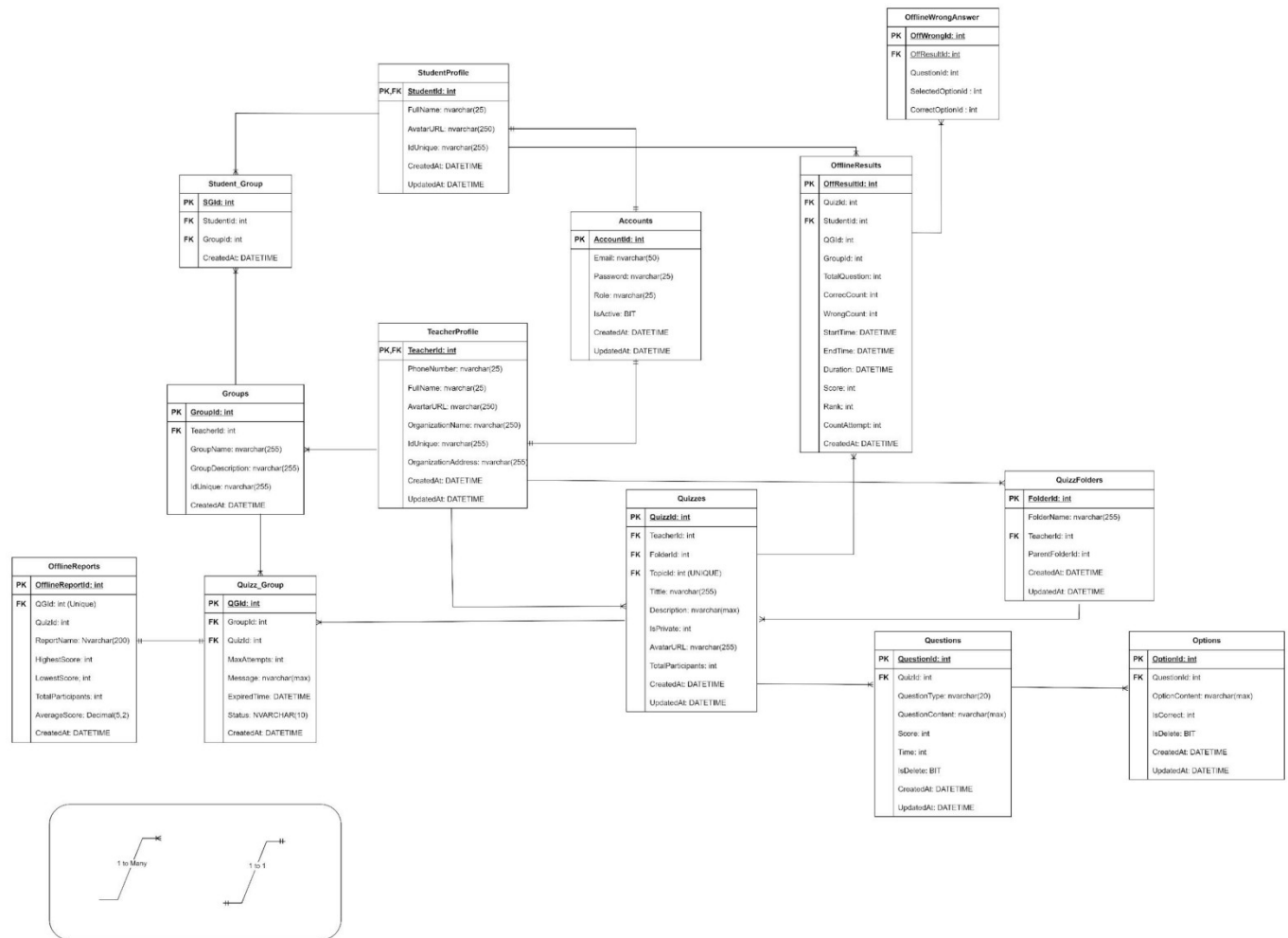
2.1 Table Overview

No.	Table name	Short Description
1	Accounts	Stores login information (email, password hash) and roles (Student, Teacher, Admin).
2	StudentProfile	Stores detailed student information (Full Name, ID Number), linked 1-to-1 with Accounts.
3	TeacherProfile	Stores detailed teacher information (Full Name, Phone), linked 1-to-1 with Accounts.
4	Groups	Stores information about groups/classes created by teachers.
5	Student_Group	A junction table (N-M) connecting students and groups.
6	QuizzFolders	Manages the folder structure for a teacher's quizzes.
7	Quizzes	Stores information about quizzes (title, description, privacy mode).
8	Quizz_Group	A junction table (N-M) to assign quizzes to groups, with deadlines and attempt limits.
9	Questions	Stores the content (MCQ, TF) and score for each QuizId.
10	Options	Stores the choices (answers) for each QuestionId and marks the correct one.
11	OfflineResults	Stores the overall result (score, time) after a student completes a quiz.
12	OfflineWrongAnswers	Details the incorrect answers from a student's quiz attempt.
13	OfflineReports	Generates a summary report (highest score, average) for an assigned quiz (QGId)

2.2 Entity Relationship Diagram



2.3 Table Relationship Diagram



2.4 Detail

2.4.1 Account

Attributes	Datatype	Null	Key	Default	Extra
AccountId	INT	Not	PK		IDENTITY(1,1)
Email	NVARCHAR(100)	Not	UNIQUE		
PasswordHash	NVARCHAR(255)	Not			
Role	NVARCHAR(20)	Not			CHECK ('Student','Teacher','Admin')
IsActive	BIT	Not		1	
CreateAt	DATETIME	Not		SYSUTCDATETIME	
UpdateAt	DATETIME	Null			

2.1.2 StudentProfile

Attributes	Datatype	Null	Key	Default	Extra
StudentId	INT	Not	PK, FK		
FullName	NVARCHAR(100)	Null			
AvatarURL	NVARCHAR(100)	Not			CHECK ('Student','Teacher','Admin')
IdUnique	NVARCHAR(100)	Not	UNIQUE		
CreateAt	DATETIME	Not		SYSUTCDATETIME	
UpdateAt	DATETIME	Null			

2.1.3 TeacherProfile

Attributes	Datatype	Null	Key	Default	Extra
TeacherId	INT	Not	PK, FK		REFERENCES Accounts(AccountId)
FullName	NVARCHAR(100)	Not			
PhoneNumber	NVARCHAR(100)	Null			
AvatarURL	NVARCHAR(100)	Null			
IdUnique	NVARCHAR(100)	Not	UNIQUE		
OrganizationName	NVARCHAR(100)	Null			
OrganizationAddress	NVARCHAR(100)	Null			
CreateAt	DATETIME	Not		SYSUTCDATETIME	
UpdateAt	DATETIME	Null			

2.1.4 Groups

Attributes	Datatype	Null	Key	Default	Extra
GroupId	INT	Not	PK		IDENTITY(1,1)
TeacherId	INT	Not	FK		REFERENCES TeacherProfile(TeacherId)
GroupName	NVARCHAR(100)	Not			
GroupDescription	NVARCHAR(100)	Null			
IdUnique	NVARCHAR(100)	Not	UNIQUE		
CreateAt	DATETIME	Not		SYSUTCDATETIME	

2.1.5 Student_Group

Attributes	Datatype	Null	Key	Default	Extra
SGId	INT	Not	PK		IDENTITY(1,1)
StudentId	INT	Not	FK		REFERENCES StudentProfile(StudentId)
GroupId	INT	Not	FK		REFERENCES Groups(GroupId)
CreateAt	DATETIME	Not		SYSUTCDATETIME	

2.1.6 QuizzFolders

Attributes	Datatype	Null	Key	Default	Extra
FolderId	INT	Not	PK		IDENTITY(1,1)
TeacherId	INT	Null	FK		REFERENCES TeacherProfile(TeacherId)
FolderName	NVARCHAR(100)	Not			
ParentFolderId	INT	Null	FK		REFERENCES QuizzFolders(FolderId)
CreateAt	DATETIME	Not		SYSUTCDATETIME	
UpdateAt	DATETIME	Null			

2.1.7 Quizzes

Attributes	Datatype	Null	Key	Default	Extra
QuizId	INT	Not	PK		IDENTITY(1,1)
TeacherId	INT	Not	FK		REFERENCES TeacherProfile(TeacherId)
FolderId	INT	Not	FK		REFERENCES QuizzFolders(FolderId)
Title	NVARCHAR(100)	Not			
Description	NVARCHAR(Max)	Not			
IsPrivate	BIT	Not			
AvatarURL	NVARCHAR(255)	Null			
TotalParticipants	INT	Not		0	
CreateAt	DATETIME	Not		SYSUTCDATETIME	
UpdateAt	DATETIME	Null			

2.1.8 Quizz_Group

Attributes	Datatype	Null	Key	Default	Extra
QGIId	INT	Not	PK		IDENTITY(1,1)
QuizId	INT	Not	FK		REFERENCES Quizzes(QuizId)
GroupId	INT	Not	FK		REFERENCES Groups(GroupId)
Message	NVARCHAR(MAX)	Null			
ExpiredTime	DATETIME	Not			
MaxAttempts	INT	Not		1	
Status	NVARCHAR(10)	Null		'Pending'	CHECK ('Pending','Completed')
CreateAt	DATETIME	Not		SYSUTCDATETIME()	

2.1.9 Questions

Attributes	Datatype	Null	Key	Default	Extra
QuestionId	INT	Not	PK		IDENTITY(1,1)
QuizId	INT	Not	FK		REFERENCES Quizzes(QuizId)
QuestionType	INT	Not			CHECK ('MCQ','TF')
QuestionContent	NVARCHAR(MAX)	Not			
Time	DATETIME	Null			
Score	INT	Null			
IsDeleted	NVARCHAR(10)	Null		0	
CreateAt	DATETIME	Not		SYSUTCDATETIME()	
UpdateAt	DATETIME	Null			

2.1.10 Options

Attributes	Datatype	Null	Key	Default	Extra
OptionId	INT	Not	PK		IDENTITY(1,1)
QuestionId	INT	Not	FK		REFERENCES Questions(QuestionId)
OptionContent	NVARCHAR(MAX)	Not			
IsCorrect	BIT	Not			
IsDeleted	BIT	Not		0	
CreateAt	DATETIME	Not		SYSUTCDATETIME()	

2.1.11 OfflineResults

Attributes	Datatype	Null	Key	Default	Extra
OffResultId	INT	Not	PK		IDENTITY(1,1)
QGId	INT	Null			
GroupId	INT	Null			
StudentId	INT	Not	FK		REFERENCES StudentProfile(StudentId)
QuizId	INT	Not	FK		REFERENCES Quizzes(QuizId)
CorrectCount	INT	Not		0	
WrongCount	INT	Not		0	
TotalQuestion	INT	Not		0	
StartDate	DATETIME	Not			
EndDate	DATETIME	Not			
Duration	INT	Not			
Score	INT	Not			
RANK	INT	Null			
CountAttempts	INT	Null		1	
CreateAt	DATETIME	Not		SYSUTCDATETIME	

2.1.12 OfflineWrongAnswers

Attributes	Datatype	Null	Key	Default	Extra
OffWrongId	INT	Not	PK		IDENTITY(1,1)
OffResultId	INT	Not	FK		REFERENCES OfflineResults(OffResultId)
QuestionId	INT	Not	FK		REFERENCES Questions(QuestionId)
SelectedOptionId	INT	Null			
CorrectOptionId	INT	Null			
CreateAt	DATETIME	Not		SYSUTCDATETIME()	

2.1.13 OfflineReports

Attributes	Datatype	Null	Key	Default	Extra
OfflineReportId	INT	Not	PK		IDENTITY(1,1)
QGId	INT	Not	FK, UNIQ		REFERENCES Quizz_Group(QGId)

			UE		
QuizId	INT	Not			
ReportName	NVARCHAR(100)	Not			
HighestScore	INT	Not			
LowestScore	INT	Not			
AverageScore	DECIMAL(5,2)	Not			
TotalParticipants	INT	Not			
CreateAt	DATETIME2	Not		SYSUTCDATETIME()	

3. Data Management and Implementation

3.1 Database design

3.1.1 Create Database and Tables

Create Database

USE VDH;

-- Table Accounts

CREATE TABLE Accounts (

 AccountId INT IDENTITY(1,1) PRIMARY KEY,

 Email NVARCHAR(100) NOT NULL UNIQUE,

 PasswordHash NVARCHAR(255) NOT NULL,

 Role NVARCHAR(20) NOT NULL CHECK (Role IN ('Student','Teacher','Admin')),

 IsActive BIT NOT NULL DEFAULT 1,

 CreateAt DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME(),

 UpdateAt DATETIME2 NULL

);

-- Table StudentProfile (1-1 với Accounts)

CREATE TABLE StudentProfile (

 StudentId INT PRIMARY KEY,

```
FullName NVARCHAR(100) NOT NULL,  
AvatarURL NVARCHAR(100) NULL,  
IdUnique NVARCHAR(100) NOT NULL UNIQUE,  
CreateAt DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME(),  
UpdateAt DATETIME2 NULL,  
FOREIGN KEY (StudentId) REFERENCES Accounts(AccountId) ON DELETE CASCADE  
);
```

-- Table TeacherProfile (1-1 với Accounts)

```
CREATE TABLE TeacherProfile (  
    TeacherId INT PRIMARY KEY,  
    FullName NVARCHAR(100) NOT NULL,  
    PhoneNumber NVARCHAR(100) NULL,  
    AvatarURL NVARCHAR(100) NULL,  
    IdUnique NVARCHAR(100) NOT NULL UNIQUE,  
    OrganizationName NVARCHAR(100) NULL,  
    OrganizationAddress NVARCHAR(100) NULL,  
    CreateAt DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME(),  
    UpdateAt DATETIME2 NULL,  
    FOREIGN KEY (TeacherId) REFERENCES Accounts(AccountId) ON DELETE CASCADE  
);
```

-- Table Groups

```
CREATE TABLE Groups (  
    GroupId INT IDENTITY(1,1) PRIMARY KEY,  
    TeacherId INT NOT NULL,  
    GroupName NVARCHAR(100) NOT NULL,  
    GroupDescription NVARCHAR(255) NULL,
```

```
IdUnique NVARCHAR(100) NOT NULL UNIQUE,  
CreateAt DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME(),  
FOREIGN KEY (TeacherId) REFERENCES TeacherProfile(TeacherId)  
);  
  
-- Table Student_Group (N-M giữa StudentProfile và Groups)  
CREATE TABLE Student_Group (  
    SGId INT IDENTITY(1,1) PRIMARY KEY,  
    StudentId INT NOT NULL,  
    GroupId INT NOT NULL,  
    CreateAt DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME(),  
    FOREIGN KEY (StudentId) REFERENCES StudentProfile(StudentId),  
    FOREIGN KEY (GroupId) REFERENCES Groups(GroupId) ON DELETE CASCADE  
);  
  
-- Table QuizzFolders  
CREATE TABLE QuizzFolders (  
    FolderId INT IDENTITY(1,1) PRIMARY KEY,  
    TeacherId INT,  
    FolderName NVARCHAR(100) NOT NULL,  
    ParentFolderId INT NULL,  
    CreateAt DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME(),  
    UpdateAt DATETIME2 NULL,  
    FOREIGN KEY (ParentFolderId) REFERENCES QuizzFolders(FolderId),  
    FOREIGN KEY (TeacherId) REFERENCES TeacherProfile(TeacherId) ON DELETE  
    CASCADE  
);  
  
-- Table Quizzes
```

```
CREATE TABLE Quizzes (  
    QuizId INT IDENTITY(1,1) PRIMARY KEY,  
    TeacherId INT NOT NULL,  
    FolderId INT NOT NULL,  
    Title NVARCHAR(100) NOT NULL,  
    Description NVARCHAR(MAX) NOT NULL,  
    IsPrivate BIT NOT NULL,  
    AvatarURL NVARCHAR(255),  
    TotalParticipants INT NOT NULL DEFAULT 0,  
    CreateAt DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME(),  
    UpdateAt DATETIME2 NULL,  
    FOREIGN KEY (TeacherId) REFERENCES TeacherProfile(TeacherId) ON DELETE CASCADE,  
    FOREIGN KEY (FolderId) REFERENCES QuizzFolders(FolderId)  
);
```

-- Table Quizz_Group (N-M giữa Quizzes và Groups)

```
CREATE TABLE Quizz_Group (  
    QGId INT IDENTITY(1,1) PRIMARY KEY,  
    QuizId INT NOT NULL,  
    GroupId INT NOT NULL,  
    Message NVARCHAR(MAX) NULL,  
    ExpiredTime Datetime2 NOT NULL,  
    MaxAttempts INT NOT NULL DEFAULT 1,  
    Status NVARCHAR(10) NOT NULL DEFAULT 'Pending' CHECK (Status IN ('Pending','Completed')),  
    CreateAt DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME(),  
    FOREIGN KEY (QuizId) REFERENCES Quizzes(QuizId) ON DELETE CASCADE,  
    FOREIGN KEY (GroupId) REFERENCES Groups(GroupId)
```


);

-- Table Questions

```
CREATE TABLE Questions (  
    QuestionId INT IDENTITY(1,1) PRIMARY KEY,  
    QuizId INT NOT NULL,  
    QuestionType NVARCHAR(10) NOT NULL CHECK (QuestionType IN ('MCQ','TF')),  
    QuestionContent NVARCHAR(MAX) NOT NULL,  
    Time INT NULL,  
    Score INT,  
    IsDeleted BIT NOT NULL DEFAULT 0,  
    CreateAt DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME(),  
    UpdateAt DATETIME2 NULL,  
    FOREIGN KEY (QuizId) REFERENCES Quizzes(QuizId) ON DELETE CASCADE  
);
```

-- Table Options

```
CREATE TABLE Options (  
    OptionId INT IDENTITY(1,1) PRIMARY KEY,  
    QuestionId INT NOT NULL,  
    OptionContent NVARCHAR(MAX) NOT NULL,  
    IsCorrect BIT NOT NULL,  
    IsDeleted BIT NOT NULL DEFAULT 0,  
    CreateAt DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME(),  
    FOREIGN KEY (QuestionId) REFERENCES Questions(QuestionId) ON DELETE  
    CASCADE  
);
```

```
CREATE TABLE OfflineResults (  

```

```
OffResultId INT IDENTITY(1,1) PRIMARY KEY,  
QGIId INT NULL,  
GroupId INT NULL,  
StudentId INT NOT NULL,  
QuizId INT NOT NULL,  
CorrectCount INT NOT NULL DEFAULT 0,  
WrongCount INT NOT NULL DEFAULT 0,  
TotalQuestion INT NOT NULL DEFAULT 0,  
StartDate DATETIME2 NOT NULL,  
EndDate DATETIME2 NOT NULL,  
Duration INT NOT NULL,  
Score INT NOT NULL,  
RANK INT NULL,  
CountAttempts INT NULL DEFAULT 1,  
CreateAt DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME(),  
FOREIGN KEY (StudentId) REFERENCES StudentProfile(StudentId),  
FOREIGN KEY (QuizId) REFERENCES Quizzes(QuizId) ON DELETE CASCADE  
);
```

-- Table OfflineWrongAnswers (Changed SelectedOptionId and CorrectOptionId to INT)

```
CREATE TABLE OfflineWrongAnswers (  
OffWrongId INT IDENTITY(1,1) PRIMARY KEY,  
OffResultId INT NOT NULL,  
QuestionId INT NOT NULL,  
SelectedOptionId INT NULL,
```

```
CorrectOptionId INT NULL,  
CreateAt DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME(),  
    FOREIGN KEY (OffResultId) REFERENCES OfflineResults(OffResultId) ON DELETE  
CASCADE,  
    FOREIGN KEY (QuestionId) REFERENCES Questions(QuestionId)  
);
```

```
CREATE TABLE OfflineReports(  
    OfflineReportId INT IDENTITY(1,1) PRIMARY KEY,  
    QGId INT NOT NULL UNIQUE,  
    QuizId INT NOT NULL,  
    ReportName NVARCHAR(100) NOT NULL,  
    HighestScore INT NOT NULL,  
    LowestScore INT NOT NULL,  
    AverageScore Decimal(5,2) NOT NULL,  
    TotalParticipants INT NOT NULL,  
    CreateAt DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME(),  
    FOREIGN KEY (QGId) REFERENCES Quizz_Group(QGId) ON DELETE CASCADE  
);
```

3.1.2 Insert Data

```
USE VDH;  
GO
```

-- 1. Insert into ACCOUNTS Table

```
INSERT INTO Accounts (Email, PasswordHash, Role) VALUES  
(N'hieuvd@dtu.edu.vn', 'hash_teacher_01', 'Teacher'),  
(N'anhtt@dtu.edu.vn', 'hash_teacher_02', 'Teacher'),  
(N'admin@vdh.system', 'hash_admin_01', 'Admin'),  
(N'tuanpm.27@dtu.edu.vn', 'hash_student_01', 'Student'),  
(N'an.nguyen@dtu.edu.vn', 'hash_student_02', 'Student'),
```

```
(N'binh.le@dtu.edu.vn', 'hash_student_03', 'Student'),  
(N'chi.pham@dtu.edu.vn', 'hash_student_04', 'Student');  
GO
```

-- 2. Insert into TeacherProfile Table

```
INSERT INTO TeacherProfile (TeacherId, FullName, IdUnique, OrganizationName)  
VALUES  
(1, N'Võ Đình Hiếu', 'TEACH-001', N'Duy Tan University);  
INSERT INTO TeacherProfile (TeacherId, FullName, IdUnique, OrganizationName)  
VALUES  
(2, N'Hồ Thị Thanh An', 'TEACH-002', N'Duy Tan University');  
GO
```

-- 3. Insert into StudentProfile Table

```
INSERT INTO StudentProfile (StudentId, FullName, IdUnique) VALUES  
(4, N'Phạm Minh Tuấn', '27211435438');  
INSERT INTO StudentProfile (StudentId, FullName, IdUnique) VALUES  
(5, N'Nguyễn Văn An', '27211112222');  
INSERT INTO StudentProfile (StudentId, FullName, IdUnique) VALUES  
(6, N'Lê Văn Bình', '27211334455');  
INSERT INTO StudentProfile (StudentId, FullName, IdUnique) VALUES  
(7, N'Phạm Thị Chi', '27211667788');  
GO
```

-- 4. Insert into QuizzFolders Table

```
INSERT INTO QuizzFolders (TeacherId, FolderName, ParentFolderId) VALUES  
(1, N'CMU-IS 401', NULL),  
(1, N'CMU-IS 201', NULL),  
(1, N'IS 401 - Midterms', 1),  
(2, N'CS 101', NULL),  
(2, N'CS 330 - AI', NULL),  
(2, N'CS 101 - Pop Quiz', 4),  
(1, N'Uncategorized', NULL);  
GO
```

-- 5. Insert into Quizzes Table

```
INSERT INTO Quizzes (TeacherId, FolderId, Title, Description, IsPrivate) VALUES  
(1, 3, N'IS 401 Midterm - Đề 1', N'Chapters 1-3.', 0),  
(1, 1, N'IS 401 - SQL DML Quiz', N'INSERT, UPDATE, DELETE.', 1),  
(2, 4, N'CS 101 - Intro to C++', N'Basic syntax and loops.', 0),  
(2, 5, N'AI - Search Algorithms', N'BFS, DFS, A*.', 0),
```

```
(1, 2, N'IS 201 - Data Structures', N'Review Stacks and Queues.', 0),  
(2, 6, N'CS 101 - Pop Quiz 1', N'Variables and Data Types.', 0),  
(1, 7, N'General Knowledge', N'A fun quiz.', 0);  
GO
```

-- 6. Insert into Questions Table

```
INSERT INTO Questions (QuizId, QuestionType, QuestionContent, Score) VALUES  
(1, 'MCQ', N'What is a Primary Key?', 50),  
(1, 'TF', N'A table can have multiple Foreign Keys.', 50),  
(2, 'MCQ', N'Which command adds new data?', 100),  
(3, 'MCQ', N'What keyword is used to declare a variable in C++?', 50),  
(3, 'TF', N'C++ requires a semicolon (;) at the end of most lines.', 50),  
(4, 'MCQ', N'Which algorithm is NOT a blind search?', 50),  
(4, 'MCQ', N'BFS stands for...?', 50),  
(5, 'MCQ', N'LIFO stands for?', 50),  
(5, 'MCQ', N'FIFO is used by which structure?', 50),  
(6, 'TF', N'In C++, "int x = 5;" is a valid declaration.', 100),  
(7, 'MCQ', N'What is the capital of Vietnam?', 50),  
(7, 'TF', N'The Earth is flat.', 50);  
GO
```

-- 7. Insert into Options Table

```
INSERT INTO Options (QuestionId, OptionContent, IsCorrect) VALUES (1, N'A unique  
identifier', 1), (1, N'A type of data', 0), (1, N'A foreign key', 0);  
INSERT INTO Options (QuestionId, OptionContent, IsCorrect) VALUES (2, N'True', 1), (2,  
N'False', 0);  
INSERT INTO Options (QuestionId, OptionContent, IsCorrect) VALUES (3, N'SELECT',  
0), (3, N'INSERT', 1), (3, N'UPDATE', 0);  
INSERT INTO Options (QuestionId, OptionContent, IsCorrect) VALUES (4, N'var', 0), (4,  
N'let', 0), (4, N'int', 1);  
INSERT INTO Options (QuestionId, OptionContent, IsCorrect) VALUES (5, N'True', 1), (5,  
N'False', 0);  
INSERT INTO Options (QuestionId, OptionContent, IsCorrect) VALUES (6, N'BFS', 0), (6,  
N'DFS', 0), (6, N'A*', 1);  
INSERT INTO Options (QuestionId, OptionContent, IsCorrect) VALUES (7, N'Breadth-  
First Search', 1), (7, N'Best-First Search', 0);  
INSERT INTO Options (QuestionId, OptionContent, IsCorrect) VALUES (8, N'Last-In,  
First-Out', 1), (8, N'First-In, First-Out', 0);  
INSERT INTO Options (QuestionId, OptionContent, IsCorrect) VALUES (9, N'Stack', 0),  
(9, N'Queue', 1);
```

```
INSERT INTO Options (QuestionId, OptionContent, IsCorrect) VALUES (10, N'True', 1),
(10, N'False', 0);
INSERT INTO Options (QuestionId, OptionContent, IsCorrect) VALUES (11, N'Ho Chi
Minh City', 0), (11, N'Da Nang', 0), (11, N'Hanoi', 1);
INSERT INTO Options (QuestionId, OptionContent, IsCorrect) VALUES (12, N'True', 0),
(12, N'False', 1);
GO
```

-- 8. Insert into Groups Table

```
INSERT INTO Groups (TeacherId, GroupName, IdUnique) VALUES
(1, N'Lớp IS 401 (Sáng T5)', 'IS401-T5'),
(1, N'Lớp IS 201 (Chiều T2)', 'IS201-T2'),
(2, N'Lớp CS 101 (Sáng T3)', 'CS101-T3'),
(2, N'Lớp CS 330 (Chiều T6)', 'CS330-T6'),
(1, N'Nhóm ôn tập SQL', 'SQL-Review'),
(2, N'Nhóm AI Club', 'AI-CLUB'),
(1, N'Lớp IS 401 (Sáng T7)', 'IS401-T7');
GO
```

-- 9. Insert into Student_Group Table

```
INSERT INTO Student_Group (StudentId, GroupId) VALUES (4, 1);
INSERT INTO Student_Group (StudentId, GroupId) VALUES (5, 1);
INSERT INTO Student_Group (StudentId, GroupId) VALUES (6, 7);
INSERT INTO Student_Group (StudentId, GroupId) VALUES (7, 3);
INSERT INTO Student_Group (StudentId, GroupId) VALUES (4, 5);
INSERT INTO Student_Group (StudentId, GroupId) VALUES (5, 5);
INSERT INTO Student_Group (StudentId, GroupId) VALUES (6, 4);
INSERT INTO Student_Group (StudentId, GroupId) VALUES (7, 6);
GO
```

-- 10. Insert into Quizz_Group Table

```
INSERT INTO Quizz_Group (QuizId, GroupId, ExpiredTime, MaxAttempts) VALUES
(1, 1, '2025-11-20 10:00:00', 1),
(1, 7, '2025-11-20 12:00:00', 1),
(2, 5, '2025-11-21 10:00:00', 3),
(3, 3, '2025-11-22 10:00:00', 1),
(4, 4, '2025-11-23 10:00:00', 1),
(4, 6, '2025-11-23 11:00:00', 2),
(7, 1, '2025-11-25 10:00:00', 1);
GO
```

-- 11. Insert into OfflineResults Table

```
INSERT INTO OfflineResults (StudentId, QuizId, QGId, GroupId, CorrectCount,
WrongCount, TotalQuestion, StartDate, EndDate, Duration, Score)
VALUES (4, 1, 1, 1, 2, 0, 2, '2025-11-18 08:00:00', '2025-11-18 08:01:30', 90, 100);
INSERT INTO OfflineResults (StudentId, QuizId, QGId, GroupId, CorrectCount,
WrongCount, TotalQuestion, StartDate, EndDate, Duration, Score)
VALUES (5, 1, 1, 1, 1, 1, 2, '2025-11-18 08:05:00', '2025-11-18 08:06:00', 60, 50);
INSERT INTO OfflineResults (StudentId, QuizId, QGId, GroupId, CorrectCount,
WrongCount, TotalQuestion, StartDate, EndDate, Duration, Score)
VALUES (6, 1, 2, 7, 0, 2, 2, '2025-11-18 08:10:00', '2025-11-18 08:11:00', 60, 0);
INSERT INTO OfflineResults (StudentId, QuizId, QGId, GroupId, CorrectCount,
WrongCount, TotalQuestion, StartDate, EndDate, Duration, Score)
VALUES (7, 3, 4, 3, 2, 0, 2, '2025-11-19 09:00:00', '2025-11-19 09:01:00', 60, 100);
INSERT INTO OfflineResults (StudentId, QuizId, QGId, GroupId, CorrectCount,
WrongCount, TotalQuestion, StartDate, EndDate, Duration, Score, CountAttempts)
VALUES (4, 2, 3, 5, 1, 0, 1, '2025-11-19 09:02:00', '2025-11-19 09:03:00', 60, 100, 1);
INSERT INTO OfflineResults (StudentId, QuizId, QGId, GroupId, CorrectCount,
WrongCount, TotalQuestion, StartDate, EndDate, Duration, Score, CountAttempts)
VALUES (5, 2, 3, 5, 0, 1, 1, '2025-11-19 09:04:00', '2025-11-19 09:05:00', 60, 0, 1);
INSERT INTO OfflineResults (StudentId, QuizId, QGId, GroupId, CorrectCount,
WrongCount, TotalQuestion, StartDate, EndDate, Duration, Score, CountAttempts)
VALUES (5, 2, 3, 5, 1, 0, 1, '2025-11-19 09:06:00', '2025-11-19 09:07:00', 60, 100, 2);
GO
```

-- 12. Insert into OfflineWrongAnswers Table

```
INSERT INTO OfflineWrongAnswers (OffResultId, QuestionId, SelectedOptionId,
CorrectOptionId)
VALUES (2, 2, 5, 4);
INSERT INTO OfflineWrongAnswers (OffResultId, QuestionId, SelectedOptionId,
CorrectOptionId)
VALUES (3, 1, 2, 1);
INSERT INTO OfflineWrongAnswers (OffResultId, QuestionId, SelectedOptionId,
CorrectOptionId)
VALUES (3, 2, 5, 4);
INSERT INTO OfflineWrongAnswers (OffResultId, QuestionId, SelectedOptionId,
CorrectOptionId)
VALUES (6, 3, 6, 7);
GO
```

-- 13. Insert into OfflineReports Table

```
INSERT INTO OfflineReports (QGIId, QuizId, ReportName, HighestScore, LowestScore,
AverageScore, TotalParticipants)
VALUES (1, 1, N'Report: IS 401 Midterm (Sáng T5)', 100, 50, 75.00, 2);
INSERT INTO OfflineReports (QGIId, QuizId, ReportName, HighestScore, LowestScore,
AverageScore, TotalParticipants)
VALUES (2, 1, N'Report: IS 401 Midterm (Sáng T7)', 0, 0, 0.00, 1);
INSERT INTO OfflineReports (QGIId, QuizId, ReportName, HighestScore, LowestScore,
AverageScore, TotalParticipants)
VALUES (3, 2, N'Report: SQL Review', 100, 0, 50.00, 2);
INSERT INTO OfflineReports (QGIId, QuizId, ReportName, HighestScore, LowestScore,
AverageScore, TotalParticipants)
VALUES (4, 3, N'Report: Intro to C++', 100, 100, 100.00, 1);
INSERT INTO OfflineReports (QGIId, QuizId, ReportName, HighestScore, LowestScore,
AverageScore, TotalParticipants)
VALUES (5, 4, N'Report: AI Search Algos', 0, 0, 0.00, 0);
INSERT INTO OfflineReports (QGIId, QuizId, ReportName, HighestScore, LowestScore,
AverageScore, TotalParticipants)
VALUES (6, 4, N'Report: AI Club Quiz', 0, 0, 0.00, 0);
INSERT INTO OfflineReports (QGIId, QuizId, ReportName, HighestScore, LowestScore,
AverageScore, TotalParticipants)
VALUES (7, 7, N'Report: General Knowledge', 0, 0, 0.00, 0);
GO
```

3.2 Procedures

3.2.2 Submit Exam (Khoa)

```
CREATE PROCEDURE usp_SubmitExam
    @StudentId INT,
    @QuizId INT,
    @Duration INT, -- Duration in seconds
    @TotalScore INT
AS
BEGIN
```



```
INSERT INTO OfflineResults (StudentId, QuizId, Score, Duration, StartDate, EndDate)
VALUES (@StudentId, @QuizId, @TotalScore, @Duration, DATEADD(SECOND, -
@Duration, SYSUTCDATETIME()), SYSUTCDATETIME());

SELECT SCOPE_IDENTITY() AS NewResultId;

END;

GO
```

3.3 Trigger

3.3.2 Automatically Update Total Participants (Khoa)

```
CREATE TRIGGER trg_UpdateTotalParticipants
ON OfflineResults
AFTER INSERT, DELETE
AS
BEGIN
UPDATE Quizzes
SET TotalParticipants = TotalParticipants + 1
FROM Quizzes q
JOIN inserted i ON q.QuizId = i.QuizId;

UPDATE Quizzes
SET TotalParticipants = TotalParticipants - 1
FROM Quizzes q
JOIN deleted d ON q.QuizId = d.QuizId;

END;

GO
```

3.4 Index

3.4.2 Composite Index for Exam History (Khoa)

```
CREATE NONCLUSTERED INDEX IX_OfflineResults_StudentHistory
```

```
ON OfflineResults(StudentId, CreateAt DESC);  
GO
```

3.5 Isolated

3.5.2 Verifying student profile while account status is being updated (Khoa)

```
CREATE OR ALTER PROCEDURE usp_VerifyStudentProfile_RepeatableRead  
    @StudentId INT  
AS  
BEGIN  
    SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;  
    BEGIN TRANSACTION;  
    SELECT  
        a.Role,  
        s.FullName,  
        a.IsActive  
    FROM  
        Accounts a  
    INNER JOIN  
        StudentProfile s ON a.AccountId = s.StudentId  
    WHERE  
        a.AccountId = @StudentId;  
    COMMIT TRANSACTION;  
END;  
GO
```

3.6 Function

3.6.2 Check Student Remaining Attempts (Khoa)

```
CREATE FUNCTION fn_CheckCanTakeQuiz (@StudentId INT, @QGId INT)
RETURNS BIT
AS
BEGIN
    DECLARE @MaxAttempts INT;
    DECLARE @CurrentAttempts INT;

    SELECT @MaxAttempts = MaxAttempts FROM Quizz_Group WHERE QGId =
    @QGId;
    SELECT @CurrentAttempts = COUNT(*)
    FROM OfflineResults
    IF @CurrentAttempts < @MaxAttempts RETURN 1;
    RETURN 0;
END;
GO
```

4. Data Integrity and Security

4.1 Role:

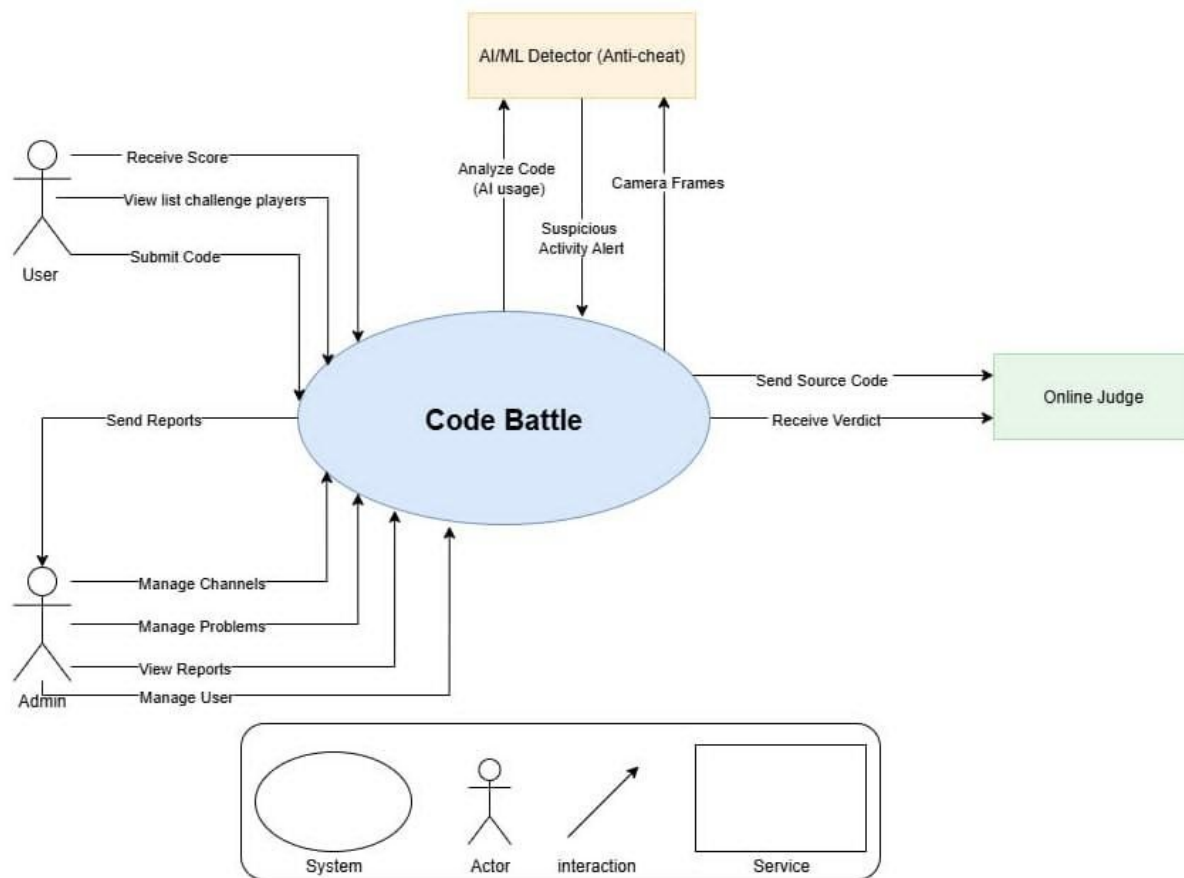


Figure 1: context diagram

Actor	Description
Admin	Manages the entire system, manages user accounts, has read/write/delete access to all data for system maintenance.
Teacher	The primary content creator role. Can create/edit/delete their own Groups, Quizzes, and Questions. Assigns quizzes to groups and views result reports.
Student	The end-user role. Can join Groups, receive assigned Quizzes, take the quizzes, and review their own results.

4.2 Data control language (DCL)

USE VDH;

GO

-- 1. Create Database Roles

CREATE ROLE db_StudentRole;

CREATE ROLE db_TeacherRole;

CREATE ROLE db_AdminRole;

GO

-- 2. Grant Permissions to Student Role

-- Students can view quiz content and submit their results.

GRANT SELECT ON OBJECT::Quizzes TO db_StudentRole;

GRANT SELECT ON OBJECT::Questions TO db_StudentRole;

GRANT SELECT ON OBJECT::Options TO db_StudentRole;

GRANT SELECT ON OBJECT::Groups TO db_StudentRole;

GRANT SELECT ON OBJECT::Student_Group TO db_StudentRole;

-- Allow students to insert their results

GRANT INSERT ON OBJECT::OfflineResults TO db_StudentRole;

GRANT INSERT ON OBJECT::OfflineWrongAnswers TO db_StudentRole;

-- Allow students to run the procedures for taking/submitting a quiz

GRANT EXECUTE ON OBJECT::sp_GetQuizDetails TO db_StudentRole;

GRANT EXECUTE ON OBJECT::sp_SubmitQuizResult TO db_StudentRole;

GO

-- 3. Grant Permissions to Teacher Role

-- Teachers have full control over their own content (Quizzes, Questions, Groups)
-- and can view student results.

```
GRANT SELECT, INSERT, UPDATE, DELETE ON OBJECT::Quizzes TO db_TeacherRole;  
GRANT SELECT, INSERT, UPDATE, DELETE ON OBJECT::Questions TO db_TeacherRole;  
GRANT SELECT, INSERT, UPDATE, DELETE ON OBJECT::Options TO db_TeacherRole;  
GRANT SELECT, INSERT, UPDATE, DELETE ON OBJECT::Groups TO db_TeacherRole;  
GRANT SELECT, INSERT, UPDATE, DELETE ON OBJECT::Student_Group TO  
db_TeacherRole;
```

-- Teachers can view reports and results

```
GRANT SELECT ON OBJECT::OfflineResults TO db_TeacherRole;  
GRANT SELECT ON OBJECT::OfflineReports TO db_TeacherRole;
```

-- Allow teachers to run procedures

```
GRANT EXECUTE ON OBJECT::sp_GetQuizDetails TO db_TeacherRole;  
GO
```

-- 4. Grant Permissions to Admin Role

-- Admins have full control over the entire database.

-- The simplest way is to add the role to the db_owner fixed role.

```
ALTER ROLE db_owner ADD MEMBER db_AdminRole;  
GO
```

5. Matrix Grade

Criteria	Weight (%)	Evaluation Criteria	GRADE
1. Database Design	25%	Accuracy of the ER diagram and database design. Detail and logic of relationships between entities and constraints.	2.5
2. Data Management and Implementation	25%	Correctness and appropriateness of SQL commands. Ability to insert correct data. Proper handling of security and data access rights.	2.5
3. Performance and Optimization	25%	Effective use of indexes and clear explanation. Proper transaction isolation levels for each procedure. Effective management and control of transactions using TCL.	2.5
4. Data Integrity and Security	15%	Data integrity and security ensured through SQL commands. Clarity and completeness of backup and recovery procedures. Triggers maintain data integrity during changes.	1.5
5. Presentation	10%	Report is structured and clearly presented. SQL commands are included and explained. Version control is accurate and up to date.	1
Total			10

6. References

[1] Table Relationship Diagram: dbdiagram.io

[2] DBMS: Microsoft SQL Server